

Flight Delay Severity Prediction Using Deep Tabular Models and a Calibrated TabTransformer

Oscar Fabrizio de Alba Gutiérrez
Artificial Intelligence Engineering
Universidad Panamericana
Aguascalientes, Mexico

Abstract—Flight delay prediction is a relevant problem in air transportation because delays affect passengers, airlines, airport operations, crew scheduling, and downstream flight planning. This paper presents a deep learning pipeline for multiclass flight delay severity prediction using real-world flight records from 2023, 2024, and 2025 obtained from the U.S. Bureau of Transportation Statistics. Approximately 21 million raw flight records were processed, and a model-ready dataset of 20,588,160 flights was produced after cleaning and feature preparation. The final target was defined as a three-class severity problem: On time, Delay, and Long delay.

The project was designed to use only schedule-based and pre-flight variables, such as carrier, origin, destination, route, distance, month, day of week, and scheduled departure information. External datasets such as weather, maintenance, air traffic restrictions, and congestion data were intentionally excluded because the objective was to evaluate whether useful delay risk patterns could be learned from stable information known before the flight. This setting is especially relevant for passengers who buy tickets weeks or months in advance, when weather or real-time operational conditions are not yet known.

Three PyTorch deep learning models were compared: an MLP with categorical embeddings, a Wide and Deep neural network, and a TabTransformer-style model with categorical and numerical feature tokens. Because the dataset is imbalanced, Macro F1-score was selected as the main evaluation metric instead of accuracy alone. The final calibrated TabTransformer achieved the best test Macro F1-score of 0.4346 and the best balanced accuracy of 0.4425. Although long-delay prediction remains difficult without external operational factors, the results show that schedule-based deep tabular models can provide useful early delay severity estimates.

Index Terms—flight delay prediction, deep learning, tabular data, TabTransformer, Wide and Deep learning, class imbalance, Macro F1-score, calibrated probabilities, PyTorch

I. INTRODUCTION

Flight delays are a persistent operational problem in commercial aviation. A delayed flight can affect passenger satisfaction, aircraft utilization, crew scheduling, airport resources, gate assignment, and connecting flights. Even when a delay is not severe, its effect can propagate through later operations and increase uncertainty for airlines and passengers. For this reason, being able to estimate delay risk before the flight takes place can support better planning and more informed decisions.

The U.S. Bureau of Transportation Statistics (BTS) commonly considers an arrival delayed when the aircraft arrives 15 minutes or more after the scheduled arrival time [2].

However, a binary delayed/not-delayed classification does not fully describe the severity of the event. A flight delayed by 20 minutes is not equivalent to a flight delayed by more than 90 minutes. The second case is more disruptive and more relevant from an operational perspective.

Many flight delay prediction studies include external variables such as weather conditions, airport congestion, air traffic restrictions, and aircraft rotation. These variables are useful and often highly predictive. However, this project follows a different direction. The main objective is not to build a model that depends on information that may only be known close to departure time. Instead, the goal is to estimate delay severity using stable information that is already known when a ticket is purchased or when an itinerary is planned.

This design is relevant because most passengers buy flights days, weeks, or even months before the travel date. At that moment, weather conditions, airport congestion, aircraft rotation, and maintenance events are unknown or would require additional forecasting systems. Therefore, a model based only on fixed or schedule-based information can still provide practical value. For example, it can help estimate whether a short connection time may be risky based on carrier, route, month, day of week, and scheduled departure characteristics.

For this reason, flight delay prediction is modeled as a three-class severity classification problem. The final target classes are:

- **On time**: arrival delay less than or equal to 15 minutes.
- **Delay**: arrival delay from 16 to 90 minutes.
- **Long delay**: arrival delay greater than 90 minutes.

The project originally considered a five-class target. However, that definition created severe imbalance, especially for very long delay classes. This made the classification problem unstable and caused early models to collapse toward either the majority class or minority classes depending on the imbalance strategy. The final three-class target was selected to preserve operational meaning while making the task more learnable.

The main challenge in this project is that delay severity is naturally imbalanced. Most flights are on time, while long delays are much less frequent. This makes accuracy an insufficient evaluation metric. A model can obtain high accuracy by predicting mostly the majority class and still fail to detect the minority Long delay class. Therefore, Macro F1-score is used as the main metric because it gives equal importance to each class.

The main contributions of this paper are:

- A complete preprocessing pipeline for a large 2023–2025 flight delay dataset.
- A leakage-safe feature set using only pre-flight or schedule-based information.
- A stratified random split designed to preserve class proportions across train, validation, and test sets.
- A comparison of three deep learning models for tabular delay severity prediction.
- A calibrated TabTransformer model that achieved the best balanced performance.
- A Python graphical interface for estimating calibrated delay probabilities from user-selected flight features.

II. STATE OF THE ART

Flight delay prediction has been studied using statistical learning, machine learning, and deep learning methods. Previous work has shown that aviation delay patterns can be learned from operational data, but prediction quality depends strongly on the information available at prediction time. Features such as previous aircraft delay, weather, airport operations, and congestion can significantly improve performance, but they may not always be available before a flight is scheduled to depart.

Machine learning approaches for flight delay prediction often use large aviation datasets and classification or regression models. Big data-driven flight delay studies have shown that machine learning can support delay forecasting and operational planning [3]. However, many highly predictive variables are also close to the actual delay event, which can create data leakage if they are used incorrectly. In this work, leakage prevention is an explicit design constraint: only pre-flight or schedule-based information is used as model input.

Weather-based flight delay prediction is common because weather is one of the most important factors affecting aviation operations. Nevertheless, the use of weather data changes the nature of the problem. Weather must also be predicted when the flight date is far in the future. For passengers or planners evaluating a flight weeks before departure, exact weather conditions are not available. Therefore, this study intentionally focuses on fixed operational variables instead of external dynamic datasets. The purpose is to estimate delay risk from information that does not depend on another forecast.

Deep learning for tabular data remains a challenging research area. Unlike images or text, tabular datasets contain mixed numerical and categorical variables with heterogeneous meanings. A common strategy is to represent high-cardinality categorical variables with learned embeddings. This is useful in this project because features such as route, origin airport, destination airport, and carrier contain many categories.

Wide and Deep learning was introduced to combine direct linear memorization with deep nonlinear generalization [4]. In this project, the Wide and Deep model was used to combine direct numerical signals with dense categorical embeddings. This allowed the model to learn both simple relationships and more complex interactions.

Transformer-based models have also been adapted to tabular data. The Transformer architecture introduced self-attention as a mechanism for learning relationships between tokens [5]. TabTransformer applies this idea to tabular data by transforming categorical embeddings into contextual representations through self-attention [6]. FT-Transformer and related studies have further evaluated Transformer-based architectures for tabular problems and shown that attention-based tabular models can be competitive under appropriate settings [7].

This project builds on these ideas by comparing an embedding-based MLP, a Wide and Deep neural network, and a TabTransformer-style architecture on a large, imbalanced, multiclass flight delay severity task using the PyTorch ecosystem [10].

III. METHODOLOGY

A. Dataset

The dataset was obtained from the Airline On-Time Performance Data database provided by the U.S. Bureau of Transportation Statistics (BTS) through the TranStats platform [1]. This database contains scheduled and actual departure and arrival times reported by certified U.S. air carriers. According to BTS, the database is available from 1987 to 2025 with monthly frequency.

Although historical records were available since 1987, only records from 2023, 2024, and 2025 were used. This decision was made for three main reasons. First, approximately 21 million raw records were already sufficient for large-scale deep learning experiments. Second, years such as 2020, 2021, and part of 2022 were affected by the COVID-19 period, which produced abnormal aviation patterns and could distort the learning process. Third, very old records may represent airline reliability, airport operations, route availability, and operational behavior that are less aligned with current aviation conditions.

In total, 36 monthly CSV files were used. The raw dataset contained approximately 21 million flight records. After cleaning and feature selection, the final model-ready dataset contained 20,588,160 flights. Table I summarizes the main dataset versions generated during the pipeline.

TABLE I
DATASET VERSIONS GENERATED DURING PREPROCESSING

Dataset	Rows	Description
Raw unified dataset	~21M	36 monthly CSV files
Model-ready dataset	20,588,160	Cleaned and selected features
Train split	14,411,712	70% stratified split
Validation split	3,088,224	15% stratified split
Test split	3,088,224	15% stratified split

Cancelled flights were removed because they did not have a valid completed arrival delay value. Rows with missing arrival delay were also removed. Negative arrival delay values were converted to zero because they indicated early arrivals, not negative delay.

The final target class distribution remained imbalanced. In the test set, the On time class represented most observations,

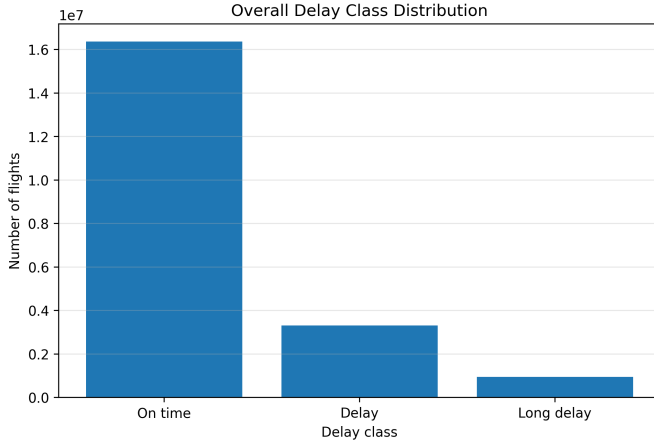


Fig. 1. Overall distribution of the final three-class delay severity target.

while Long delay remained the minority class. Table II shows the final test support used for evaluation.

TABLE II
TEST SET CLASS DISTRIBUTION

Class	Label	Test Support
0	On time (≤ 15 min)	2,453,607
1	Delay (16–90 min)	494,801
2	Long delay (> 90 min)	139,816

The overall distribution of the final three-class target is shown in Fig. 1. The figure illustrates the strong dominance of the On time class and the smaller proportion of Long delay cases.

B. Target

The target variable was created from arrival delay minutes. Let d_i be the cleaned arrival delay in minutes for flight i , where early arrivals were set to zero. The target y_i was defined as:

$$y_i = \begin{cases} 0, & d_i \leq 15 \\ 1, & 16 \leq d_i \leq 90 \\ 2, & d_i > 90 \end{cases} \quad (1)$$

This target definition balanced operational meaning and statistical stability. The first threshold followed the standard 15-minute delay definition used in aviation reporting [2]. The second threshold separated common manageable delays from longer disruptions.

C. Split

A stratified random split was used. The final split proportions were:

- **Train:** 70%
- **Validation:** 15%
- **Test:** 15%

The split was stratified by the target class. This was done to preserve class proportions across all subsets, which was

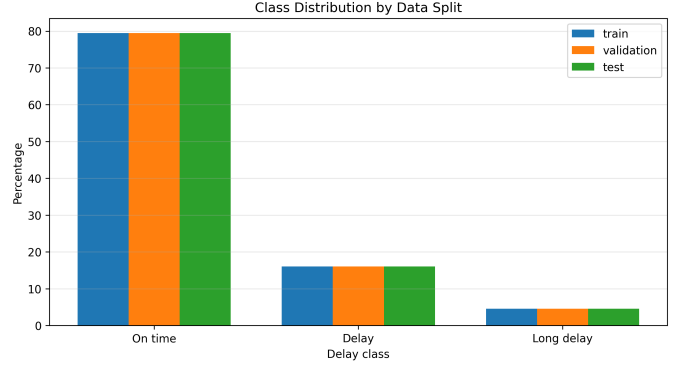


Fig. 2. Class distribution across the stratified train, validation, and test splits.

important because the dataset was imbalanced. Since the dataset included three complete years of flight records, the stratified random split also helped represent seasonal patterns from different years across training, validation, and test sets.

Table III summarizes the final split structure.

TABLE III
STRATIFIED SPLIT STRUCTURE

Split	Percentage	Records
Train	70%	14,411,712
Validation	15%	3,088,224
Test	15%	3,088,224

The class distribution across the stratified train, validation, and test subsets is shown in Fig. 2. Similar proportions across subsets were expected because stratification was applied.

D. Features

The feature engineering stage produced two groups of model inputs: categorical features and numerical features.

The categorical features were:

- operating carrier,
- origin airport,
- origin state,
- destination airport,
- destination state,
- route,
- departure time block,
- arrival time block.

The numerical features were:

- distance,
- scheduled departure minutes of day,
- sine and cosine transformations of scheduled departure time,
- sine and cosine transformations of month,
- sine and cosine transformations of day of week,
- weekend indicator.

Table IV summarizes the feature groups used for training. Categorical features were encoded as integer IDs and used with embedding layers. Encoders were fitted only on the training set. Unknown validation or test categories were mapped to

TABLE IV
FEATURE GROUPS USED FOR TRAINING

Type	Features
Categorical	Carrier, origin, origin state, destination, destination state, route, departure time block, arrival time block
Numerical	Distance, scheduled departure minutes, cyclic departure time features, cyclic month/day features, weekend indicator

zero. Numerical features were scaled using a StandardScaler fitted only on the training data.

Time variables were transformed cyclically. For example, scheduled departure time was converted to minutes of day and then encoded as:

$$x_{\sin} = \sin\left(2\pi\frac{m}{1440}\right), \quad (2)$$

$$x_{\cos} = \cos\left(2\pi\frac{m}{1440}\right), \quad (3)$$

where m is the number of minutes since midnight. This representation avoided treating 23:59 and 00:01 as far apart.

E. Cardinality

High-cardinality categorical variables are difficult to represent with one-hot encoding. The route feature had 7,454 unique categories, while origin and destination each had 363 categories. Embeddings provided a compact dense representation and allowed the model to learn similarities between routes, airports, and carriers.

Table V summarizes the cardinality of the categorical features used by the embedding-based models.

TABLE V
CATEGORICAL FEATURE CARDINALITIES

Feature	Cardinality
ROUTE	7,454
ORIGIN	363
DEST	363
ORIGIN_STATE_ABR	53
DEST_STATE_ABR	53
DEP_TIME_BLK	20
ARR_TIME_BLK	20
OP_UNIQUE_CARRIER	16

The cardinality differences between categorical variables are shown in Fig. 3. The route feature was the highest-cardinality feature, which supported the use of embeddings.

F. Leakage

A key methodological decision was to avoid data leakage. The model was designed to predict delay severity using only information that would be available before the flight or from the flight schedule. Therefore, post-flight or delay-cause columns were excluded from the model features.

Excluded columns included actual departure time, departure delay, arrival delay, taxi time, air time, carrier delay, weather

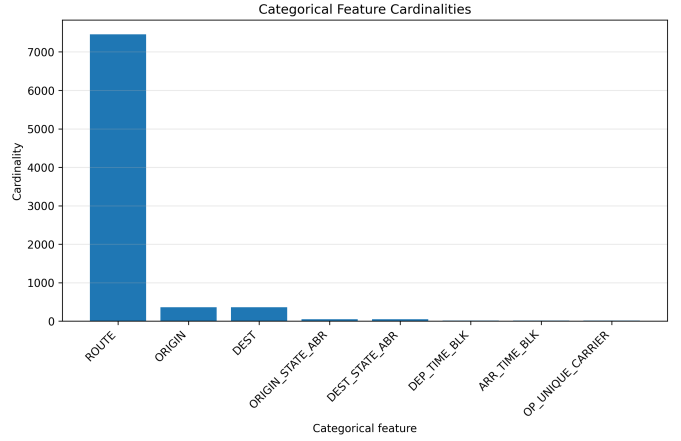


Fig. 3. Cardinality of categorical features used by the embedding-based models.

delay, NAS delay, late aircraft delay, security delay, cancellation code, and cleaned arrival delay minutes. These variables would make the model appear much stronger, but they were not valid for a pre-flight prediction setting.

The exclusion of external datasets was also intentional. Weather, congestion, air traffic restrictions, and maintenance data can be highly informative, but these variables are not fixed long before the flight. Since this project was designed as an early risk estimator, only stable information was preserved.

G. Class Weights

Class weights were calculated from the training split to reduce the effect of class imbalance. These weights helped the models pay more attention to Delay and Long delay examples. However, very aggressive weighting or oversampling was found to destabilize training in early experiments. The final models used controlled and softened class-weight transformations rather than extreme sampling strategies.

IV. MODELS

A. MLP

The first model was an MLP with categorical embeddings. Each categorical variable was passed through an embedding layer. The embeddings were concatenated with 9 scaled numerical features and passed through a feedforward network.

The hidden layers had sizes 256, 128, and 64. Batch normalization, ReLU activations, and dropout of 0.3 were applied. The MLP served as the baseline neural architecture.

The MLP used AdamW optimization, a learning rate of 0.001, weight decay of 10^{-4} , batch size 4096, and CrossEntropyLoss with softened class weights. The model was trained for 10 epochs.

B. Wide and Deep

The second model was a Wide and Deep neural network. It had two paths:

- a wide path that mapped numerical features directly to output logits,

- a deep path that processed categorical embeddings and numerical features through dense layers.

The deep path used hidden layers of 384, 192, and 96 units. The final logits were computed as the sum of the wide and deep logits. This architecture was designed to combine simple direct numerical relationships with nonlinear feature interactions.

The Wide and Deep model used AdamW, learning rate 0.001, weight decay 10^{-4} , batch size 4096, and controlled class weights. It was trained for 10 epochs.

C. TabTransformer

The third model was a TabTransformer / FT-Transformer style architecture. It represented tabular features as tokens:

- categorical tokens from embedding layers,
- numerical tokens from learned numerical projections,
- one CLS token for global representation.

The token sequence was processed by a Transformer Encoder with:

- $d_{model} = 96$,
- 6 attention heads,
- 4 encoder layers,
- feedforward dimension 384,
- GELU activation,
- transformer dropout 0.12.

The final representation combined the CLS token output with mean token pooling. This representation was concatenated with the numerical features and passed to a classification head with hidden layers of 384 and 192. This pooling strategy was used to avoid relying only on the CLS token and to capture information from the full set of feature tokens.

Training used AdamW [8], OneCycleLR, mixed precision, gradient clipping, label smoothing, and early stopping. Probability calibration was applied after training using the validation set following traditional optimization techniques [9]. This calibration multiplied class probabilities by validation-tuned class multipliers and then renormalized the result. The best multipliers were [1.0, 1.16, 1.305]. Calibration was not tuned on the test set.

V. EXPERIMENTS

All experiments were run on an HP OMEN 16 laptop with an NVIDIA GeForce RTX 4060 Laptop GPU, an Intel Core i9-13900HX CPU, and 32 GB of DDR5 5600 MHz RAM. The batch size was 4096 for all models.

Table VI summarizes the training configuration and time. Training time was reported because model complexity and computational cost are important in practical deep learning workflows.

Accuracy, Weighted F1-score, Macro F1-score, and Balanced Accuracy were considered. Accuracy is useful but can be misleading under imbalance. Weighted F1-score is affected by class support and is therefore also influenced by the majority class. Macro F1-score gives equal weight to each class and was selected as the main metric.

TABLE VI
TRAINING CONFIGURATION AND TIME

Model	Epochs	Time (min)
MLP with Embeddings	10	59.90
Wide and Deep	10	54.49
TabTransformer	12	175.52

For a class c , precision and recall are:

$$Precision_c = \frac{TP_c}{TP_c + FP_c}, \quad (4)$$

$$Recall_c = \frac{TP_c}{TP_c + FN_c}. \quad (5)$$

The F1-score for class c is:

$$F1_c = 2 \cdot \frac{Precision_c \cdot Recall_c}{Precision_c + Recall_c}. \quad (6)$$

Macro F1 is the average F1 across all classes:

$$MacroF1 = \frac{1}{C} \sum_{c=1}^C F1_c. \quad (7)$$

Because $C = 3$ in this project, each class contributes equally to the final Macro F1-score.

VI. RESULTS

A. Overall Performance

Table VII presents the final test results. The raw TabTransformer achieved the highest accuracy and weighted F1-score. However, the calibrated TabTransformer obtained the best Macro F1-score and balanced accuracy, so it was selected as the final model.

TABLE VII
FINAL TEST PERFORMANCE

Model	Acc.	Bal. Acc.	Macro F1	Weighted F1
MLP with Embeddings	0.7162	–	0.4192	0.7179
Wide and Deep	0.6539	–	0.4149	0.6817
TabTransformer Raw	0.7344	0.4259	0.4309	0.7284
TabTransformer Calibrated	0.7057	0.4425	0.4346	0.7155

The Macro F1-score comparison is shown in Fig. 4. The calibrated TabTransformer achieved the highest Macro F1-score, while the raw TabTransformer achieved the highest accuracy.

The results show an important trade-off. The MLP achieved a strong baseline and obtained better Macro F1 than the Wide and Deep model. The Wide and Deep model predicted delayed classes more often, but this reduced its accuracy and did not improve Macro F1 over the MLP. The raw TabTransformer achieved the best accuracy, while the calibrated TabTransformer achieved the best Macro F1 and balanced accuracy.

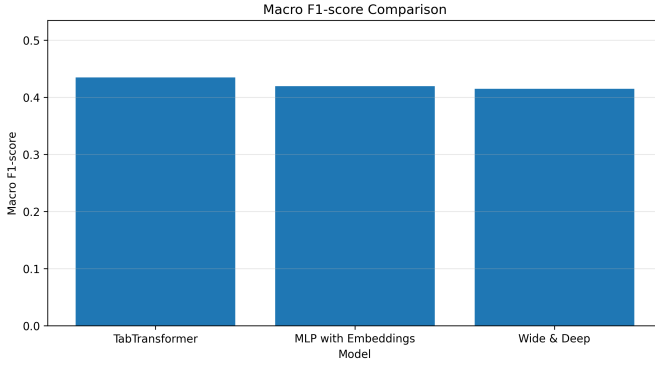


Fig. 4. Macro F1-score comparison of the evaluated models.

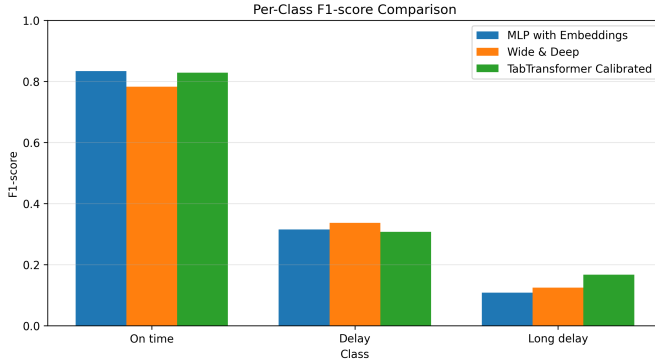


Fig. 5. Per-class F1-score comparison across the main models.

B. Per-Class Performance

Table VIII shows per-class F1-scores. The On time class was the easiest class for all models. Delay was more difficult, and Long delay was the most difficult class. This was expected because Long delay was the minority class and was likely influenced by external operational conditions not present in the feature set.

TABLE VIII
PER-CLASS F1-SCORE ON THE TEST SET

Model	On time	Delay	Long delay
MLP	—	—	—
Wide and Deep	0.78	0.34	0.12
TabTransformer Raw	0.85	0.30	0.15
TabTransformer Calibrated	0.83	0.31	0.17

The per-class F1-score comparison is shown in Fig. 5. The calibrated TabTransformer obtained the best Long delay F1-score.

Table IX shows the per-class recall values for the main reported models. The calibrated TabTransformer increased Long delay recall compared with the raw TabTransformer, improving from 0.13 to 0.18. This improvement was important because Long delay was the most difficult and operationally relevant minority class.

TABLE IX
PER-CLASS RECALL ON THE TEST SET

Model	On time	Delay	Long delay
Wide and Deep	0.72	0.49	0.10
TabTransformer Raw	0.86	0.29	0.13
TabTransformer Calibrated	0.81	0.33	0.18

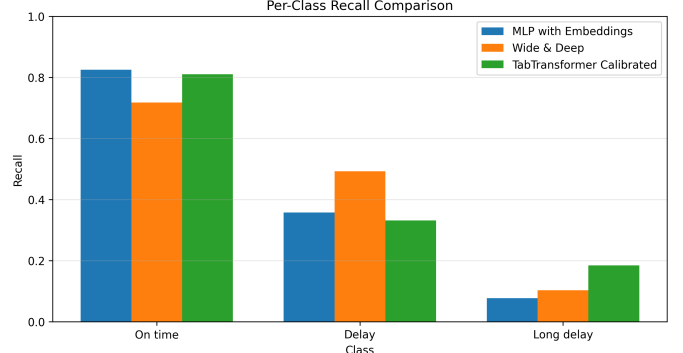


Fig. 6. Per-class recall comparison across the main models.

The per-class recall comparison is shown in Fig. 6. The figure shows that calibration increased Long delay recall at the cost of lower overall accuracy.

C. Confusion Matrix

The confusion matrix analysis shows that Long delay remained the most difficult class. Many Long delay flights were still classified as On time or Delay. This was a limitation of the current feature set. However, the calibrated TabTransformer predicted more Delay and Long delay cases than the raw version, which improved Macro F1 and balanced accuracy.

The normalized confusion matrix for the calibrated TabTransformer is shown in Fig. 7.

D. Training Behavior

The TabTransformer was trained for 12 epochs and achieved its best raw validation Macro F1-score of 0.4314. Training was more expensive than the MLP and Wide and Deep models because the Transformer Encoder processed categorical and numerical feature tokens with self-attention.

The validation Macro F1-score curves are shown in Fig. 8. These curves helped compare the learning behavior of the three architectures.

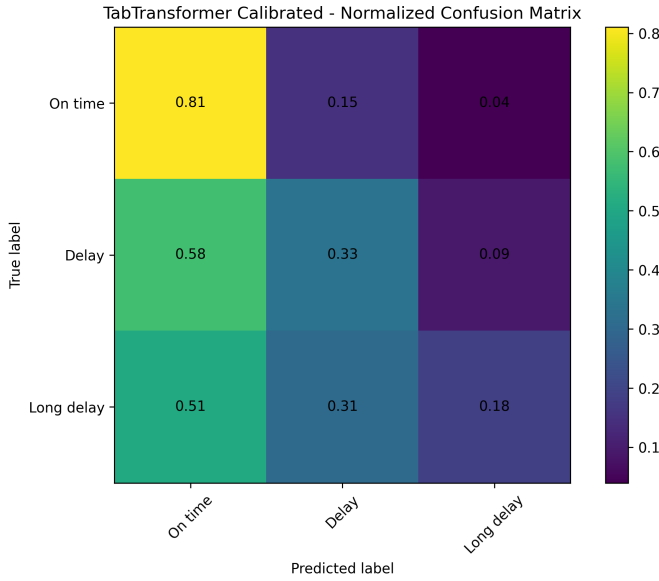


Fig. 7. Normalized confusion matrix for the calibrated TabTransformer.

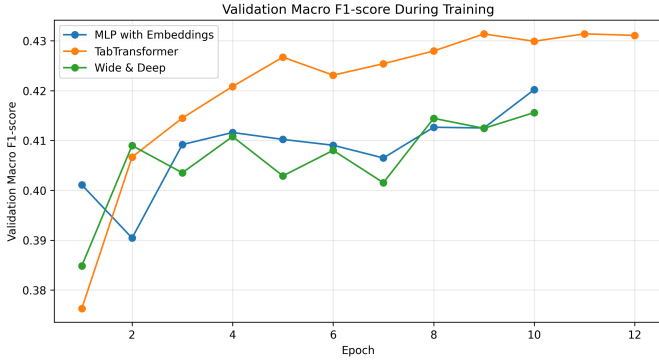


Fig. 8. Validation Macro F1-score during training.

VII. PROTOTYPE

A small Python graphical application was implemented to demonstrate how the trained model could be used for inference. The application loaded the calibrated TabTransformer checkpoint, metadata, categorical encoders, numerical scaler, and route lookup file. Flight information was selected from dropdown lists, including carrier, route, month, day of week, scheduled departure time, and arrival time block.

The interface displayed calibrated probabilities for the three delay severity classes. This made the model easier to interpret than only returning a hard class prediction. The prototype was not a production system, but it showed how the trained model could be converted into a practical decision-support tool.

The graphical inference interface is shown in Fig. 9.

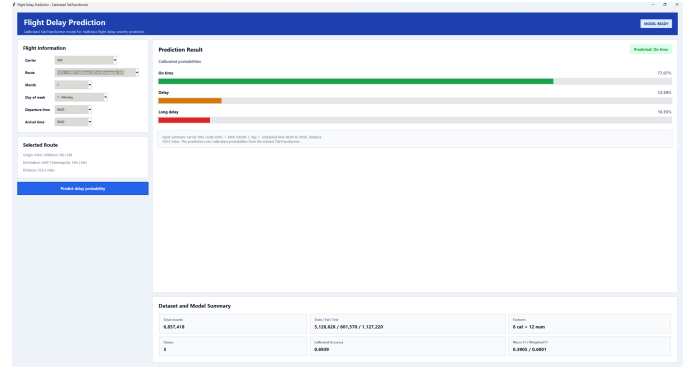


Fig. 9. Graphical inference interface developed in Python to estimate calibrated delay severity probabilities using the trained TabTransformer model.

VIII. DISCUSSION

The experimental results should be interpreted according to the difficulty and constraints of the task. The model only uses information available before the flight or from the schedule, such as airline, route, origin, destination, scheduled departure time, arrival time block, distance, and calendar features. It does not use weather, aircraft rotation, previous flight delay, maintenance events, airport congestion, air traffic control restrictions, or real-time operational disruptions.

This limitation was intentional. The purpose of the project was not to compete with models that use all possible operational signals. Instead, the project was designed to explore whether useful delay severity estimates can be obtained from information that is already known far in advance. In that sense, the results are meaningful because they were obtained without relying on external variables that would require separate forecasts or real-time systems.

The relatively low Long delay recall is expected. Long delays are often caused by events that are not visible from schedule information alone. Weather disruptions, crew issues, aircraft rotation problems, maintenance events, and air traffic restrictions can appear close to the departure date or even during the same day. Therefore, it would be unrealistic to expect schedule-only features to perfectly predict those cases.

However, the calibrated TabTransformer still improved Long delay recall from 0.13 to 0.18 compared with the raw version. This is not a high recall value in absolute terms, but it is relevant under the constraints of the project. The model was able to identify additional long-delay risk patterns without using weather, maintenance, or real-time airport information.

The stratified random split helped preserve class proportions and seasonal variability across training, validation, and test sets. This was useful after extending the dataset to three years because each subset included examples from different seasons and operational periods. A purely temporal split could be considered in future work if the objective is strict chronological deployment evaluation.

The model comparison shows that accuracy alone is not enough. The raw TabTransformer achieved the best accuracy and weighted F1-score, but the calibrated TabTransformer

achieved better Macro F1 and balanced accuracy. This means that calibration reduced the dominance of the majority class and improved minority-class behavior. In an imbalanced task, this is more important than maximizing accuracy only.

The improvement over the MLP and Wide and Deep models is moderate, not dramatic. However, it is meaningful because it occurs in the most difficult part of the problem: minority-class delay severity detection. In practical itinerary planning, missing a Long delay can be more costly than misclassifying an On time flight as a moderate Delay risk. Therefore, selecting the best model by Macro F1 is more appropriate than selecting only by accuracy.

IX. CONCLUSION

A complete deep learning pipeline for flight delay severity prediction using 2023–2025 flight records was presented. The final model-ready dataset contained 20,588,160 flights. A three-class target was used to classify flights as On time, Delay, or Long delay. This target definition improved stability compared with the original five-class design, which produced severe imbalance.

Three PyTorch models were evaluated: an MLP with embeddings, a Wide and Deep neural network, and a TabTransformer-style architecture. The MLP provided a strong baseline with a Macro F1-score of 0.4192. The Wide and Deep model predicted delayed classes more often, but it did not outperform the MLP in Macro F1. The raw TabTransformer achieved the best accuracy, while the calibrated TabTransformer achieved the best Macro F1-score of 0.4346 and the best balanced accuracy of 0.4425. Therefore, the calibrated TabTransformer was selected as the final model.

The results are meaningful because direct data leakage was avoided and only schedule-based or pre-flight information was used. Although Long delay prediction remains difficult, the model provides a useful early risk estimate under realistic information constraints. Future work can integrate weather data, aircraft rotation information, previous flight delay, airport

congestion indicators, maintenance events, and real-time air traffic conditions. Additional experiments can also compare deep learning models against gradient boosting baselines, evaluate repeated runs with confidence intervals, and explore cost-sensitive thresholds for operational decision-making.

Finally, the graphical inference prototype can be extended into a web dashboard or operational monitoring tool. This would make the model easier to test with different flight scenarios and could support practical delay risk estimation.

REFERENCES

- [1] Bureau of Transportation Statistics, U.S. Department of Transportation, “Airline On-Time Performance Data,” TranStats Database Profile. [Online]. Available: https://transtats.bts.gov/DatabaseInfo.asp?QQ_VQ=EFD&DB_URL=. Accessed: Apr 2026.
- [2] Bureau of Transportation Statistics, U.S. Department of Transportation, “Airline On-Time Statistics and Delay Causes.” [Online]. Available: https://www.transtats.bts.gov/ot_delay/ot_delaycause1.asp. Accessed: Apr 2026.
- [3] J. Huo, H. Zhang, and V. A. Profillidis, “The prediction of flight delay: A big data-driven machine learning approach,” in *Proc. International Conference on Transportation and Development*, 2020.
- [4] H.-T. Cheng, L. Koc, J. Harmsen, T. Shaked, T. Chandra, H. Aradhye, G. Anderson, G. Corrado, W. Chai, M. Ispir, R. Anil, Z. Haque, L. Hong, V. Jain, X. Liu, and H. Shah, “Wide & Deep learning for recommender systems,” in *Proc. 1st Workshop on Deep Learning for Recommender Systems*, 2016, pp. 7–10.
- [5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [6] X. Huang, A. Khetan, M. Cvitkovic, and Z. Karnin, “TabTransformer: Tabular data modeling using contextual embeddings,” arXiv preprint arXiv:2012.06678, 2020.
- [7] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, “Revisiting deep learning models for tabular data,” in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 18932–18943.
- [8] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations*, 2019.
- [9] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. 34th International Conference on Machine Learning*, 2017, pp. 1321–1330.
- [10] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al., “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019.